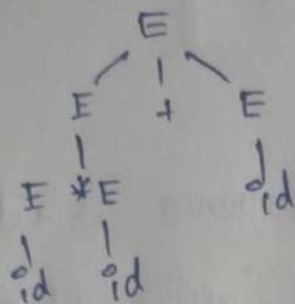


Short Answer Test

- 1) The parser checks whether the token sequence $id + id * id$ follows grammar rules. It builds a parse tree, applies operator precedence and accepts the expressions if it is syntactically correct.
- 2) Grammar: $S \rightarrow asb | \epsilon$
Derivation for $aaabbb$:
$$\begin{aligned} S &\rightarrow asb \\ &\rightarrow aaSbb \\ &\rightarrow aaasbbb \\ &\rightarrow aaa\epsilon bbb \\ &\rightarrow aaabbb \end{aligned}$$

 $\therefore$ Hence $aaabbb$ belongs to the language.
- 3) Remove left recursion: $E \rightarrow E + T \mid T$
Result:
$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \end{aligned}$$
- 4) Grammar: $A \rightarrow aB \mid \epsilon$
 $B \rightarrow b$
 $FIRST(A) = \{a, \epsilon\}$
- 5) Grammar: $A \rightarrow Aa \mid b$
Not suitable for top-down parsing because it has left recursion.
- 6) Left factoring: $S \rightarrow \text{while } E \text{ do } S \mid \text{while } E \text{ do } S \text{ else } S$
Result:
$$\begin{aligned} S &\rightarrow \text{while } E \text{ do } SS' \\ S' &\rightarrow \text{else } S \mid \epsilon \end{aligned}$$

7) Parse tree for $id * id + id$:



8) Grammar: $S \rightarrow aA$
 $A \rightarrow bA \mid c$

SC) { match(a); AC(); }

AC) {
 if (next == 'b') {
 match(b);
 AC();
 }
 else
 match(c);
}

9) Grammar: $S \rightarrow AB$
 $A \rightarrow a \mid \epsilon$
 $B \rightarrow b$

	a	b
S	AB	AB
A	a	ϵ
B	-	b

10) First three actions for $id + id$:

- 1) Shift id
- 2) Reduce $E \rightarrow id$
- 3) Shift +

11) Handle of $id * id + id$:

Handle = id (first reduction)

12) Operator precedence: $* > +$

Operator precedence parser first shifts operands, then reduces using higher precedence operators and accepts if stack reduce to Start Symbol

LR parser: A bottom-up parser that scans input from left to right and produces the rightmost derivation in reverse

Ex: Shift id $\rightarrow$ Reduce $E \rightarrow id \rightarrow$ Accept

Steps: for SLR table:

- 1) Augmented grammar
- 2) Construct LR(0) items
- 3) Compute FIRST and FOLLOW
- 4) Build ACTION and GOTO tables

Grammar:

$S \rightarrow AB$

$A \rightarrow a | \epsilon$

$B \rightarrow b$

$\text{Follow}(A) = \{b\}$

Augmented grammar:

$S' \rightarrow S$

$S \rightarrow aA$

$A \rightarrow b$

Closure of LR(0) item:

$S' \rightarrow \cdot S$

$S \rightarrow \cdot aA$

$S \rightarrow \cdot b$

Grammar:

$E \rightarrow E + E \mid id$

Ambiguous because $id + id + id$ has more than one parse tree

Leftmost derivation for $id + id * id$:

$E \rightarrow E + E$

$\rightarrow id + E$

$\rightarrow id + E * E$

$\rightarrow id + id * E$

$\rightarrow id + id * id$

21) Left factoring:

$$S \rightarrow aBC \mid aBD \mid b$$

Result:

$$S \rightarrow aBX \mid b$$
$$X \rightarrow C \mid D$$

22) Grammar:

$$F \rightarrow (E) \mid id$$

$$FIRST(F) = \{ (, id \}$$

23) Grammar:

$$S \rightarrow aA \mid aB$$

$$A \rightarrow b$$

$$B \rightarrow c$$

Not suitable for predictive parsing because both productions start with a (FIRST/FIRST Conflict)

24) Recursive descent procedure

```
SC() {  
    match(a);  
    match(b);  
    SC();  
}
```

25) Grammar:

$$S \rightarrow aS \mid b$$

Predictive parsing table:

	a	b
S	$S \rightarrow aS$	$S \rightarrow b$